

Manual rápido: de una aplicación a un dominio público, con Docker y Cloudflare Tunnel

Receta repetible para cualquier aplicación, no sólo GestIA. Lo que hace cada parte:

- **Docker** levanta la aplicación y la deja escuchando en un puerto de la máquina.
- **Cloudflare Tunnel** conecta ese puerto con un nombre de internet. No son órdenes de Docker: son órdenes de `cloudflared`, o tres clics en el panel.
- **No se abre ningún puerto en el módem ni se configura DNS a mano.** El túnel sale hacia afuera, y el registro de DNS lo crea Cloudflare solo.

Tiempo real la primera vez: media hora. Las siguientes, diez minutos.

Parte 0 · Lo que necesitas antes

1. **Docker Desktop** instalado y corriendo.
 2. **Una cuenta de Cloudflare** (gratuita sirve).
 3. **Un dominio.** Se compra en la Parte 2; si ya tienes uno, sólo hay que pasarlo a Cloudflare.
-

Parte 1 · Empaquetar y levantar con Docker

1.1 Un Dockerfile por pieza

Un portal web estático o de framework (Angular, React, Vue) se construye y se sirve con nginx:

```
# Dockerfile del portal
FROM node:24-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . ./
RUN npm run build

FROM nginxinc/nginx-unprivileged:alpine AS runtime
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build --chown=101:101 /app/dist/<tu-salida>/browser/ /usr/share/nginx/html/
EXPOSE 8080
HEALTHCHECK --interval=10s --timeout=3s --retries=5 \
  CMD wget -q --spider http://127.0.0.1:8080/healthz || exit 1
```

Una API (aquí .NET; el patrón es el mismo con Node, Python o Java) se construye en una imagen y se ejecuta en otra más chica:

```
FROM mcr.microsoft.com/dotnet/sdk:10.0-alpine AS build
WORKDIR /src
COPY . ./
RUN dotnet publish MiApi.csproj -c Release -o /app/publish

FROM mcr.microsoft.com/dotnet/aspnet:10.0-alpine AS runtime
WORKDIR /app
ENV ASPNETCORE_HTTP_PORTS=8080
COPY --from=build /app/publish ./
USER $APP_UID
EXPOSE 8080
HEALTHCHECK --interval=10s --timeout=3s --retries=5 \
    CMD wget -q --spider http://127.0.0.1:8080/health/ready || exit 1
ENTRYPOINT ["dotnet", "MiApi.dll"]
```

1.2 Que el portal hable con la API por el mismo nombre

Es el truco que evita CORS, direcciones incrustadas en el paquete y un segundo nombre de dominio: nginx sirve el portal **y** reenvía `/api/` a la API.

```

# nginx.conf
server {
    listen 8080;
    root /usr/share/nginx/html;
    index index.html;

    # La politica de seguridad. Agrega aqui CADA origen externo que uses:
    # mapas, tipografias, imagenes de otro sitio. Si no esta en la lista, el navegador lo bloquea
    # y en desarrollo no se nota, porque ahi no hay politica.
    add_header Content-Security-Policy "default-src 'self'; img-src 'self' data:; script-src
'self'; style-src 'self' 'unsafe-inline'; connect-src 'self'; frame-ancestors 'none'" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;

    location = /healthz { return 200 "healthy\n"; }

    location /api/ {
        proxy_pass http://api:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location / { try_files $uri $uri/ /index.html; }
}

```

Y en el código del portal, las llamadas van en relativo: `/api/v1/...`, nunca `http://localhost:8080`.

1.3 El compose

```
name: mi-app

services:
  db:
    image: postgres:17-alpine          # o mcr.microsoft.com/mssql/server:2025-latest
    environment:
      POSTGRES_PASSWORD: "${DB_PASSWORD:?falta DB_PASSWORD en .env}"
    ports:
      - "127.0.0.1:5432:5432"          # sólo esta máquina, no la red de la casa
    volumes:
      - datos:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      retries: 12
    restart: unless-stopped

  api:
    image: mi-app/api:local
    build:
      context: .
      dockerfile: api/Dockerfile
    depends_on:
      db: { condition: service_healthy }
    environment:
      ConnectionStrings__Default: "Host=db;Password=${DB_PASSWORD}"
    ports:
      - "127.0.0.1:8080:8080"
    restart: unless-stopped

  portal:
    image: mi-app/portal:local
    build:
      context: .
      dockerfile: portal/Dockerfile
    depends_on:
      api: { condition: service_healthy }
    ports:
      - "127.0.0.1:4200:8080"          # ÉSTE es el puerto que va a publicar el túnel
    restart: unless-stopped
```

```
volumes:
  datos:
```

Tres detalles que se pagan caros si faltan:

- ****restart: unless-stopped**** en todo lo que deba sobrevivir a un reinicio. Sin esto, reinicias la máquina y el sitio queda caído hasta que entres a levantarlo a mano.
- ****127.0.0.1: delante del puerto.**** Sin eso, la base y la API quedan visibles para cualquiera en la misma red. El túnel igual funciona, porque entra por **localhost**.
- ****Las contraseñas en .env**, y .env en .gitignore**. Nunca dentro del compose.

1.4 Levantar y comprobar

```
Copy-Item .env.example .env      # y ponle contraseñas de verdad
docker compose build
docker compose up -d
docker compose ps                # todos "healthy"
curl http://localhost:4200       # el portal responde
```

Hasta aquí la aplicación sólo existe en tu máquina. Eso es correcto.

Parte 2 · El dominio

Opción A — comprarlo en Cloudflare (lo más simple, porque ya queda configurado):

dash.cloudflare.com → *Domain Registration* → *Register Domain* → buscas el nombre y pagas.

Opción B — ya lo tienes en otro registrador: en Cloudflare, *Add a site*, escribes el dominio, eliges el plan gratuito, y Cloudflare te da **dos servidores de nombres**. Entrás al panel de tu registrador y reemplazas los suyos por esos dos. Tarda entre minutos y unas horas en quedar activo.

No hace falta crear ningún registro a mano: eso lo hace el túnel en la Parte 3.

Parte 3 · El túnel

Dos caminos. Elige uno.

Camino 1 · Desde el panel (el que usa GestIA hoy)

1. one.dash.cloudflare.com → **Networks** → **Tunnels** → **Create a tunnel** → *Cloudflared*.
2. Le pones nombre (por ejemplo **mi-app**) y guardas.
3. Te muestra una orden para instalar. En Windows queda así:

```
cloudflared.exe service install <EL-TOKEN-QUE-TE-DIO>
```

Eso deja un **servicio de Windows** con arranque automático, que es lo que hace que el túnel vuelva solo después de un reinicio.

1. En la pestaña **Public Hostname**, agregas:

- *Subdomain*: `dev` — *Domain*: `mi-dominio.com`
- *Service*: `HTTP` → `localhost:4200`

Cuidado con el subdominio: en *Subdomain* va sólo `dev`, no `dev.mi-dominio.com`. Si escribes el nombre completo acabas con un registro `dev.mi-dominio.com.mi-dominio.com`, que no sirve para nada y luego hay que borrar.

1. El registro de DNS lo crea Cloudflare solo, de tipo *Tunnel* y con proxy encendido.

La contrapartida de este camino: **las reglas viven en el panel, no en tu repositorio**. Anota en un documento a qué puerto local entra cada nombre.

Camino 2 · Desde la línea de órdenes (queda versionado)

```
# 1. Autorizar esta máquina contra tu cuenta (abre el navegador y eliges el dominio)
cloudflared tunnel login

# 2. Crear el túnel. Deja un archivo de credenciales <UUID>.json en %USERPROFILE%\cloudflared\
cloudflared tunnel create mi-app

# 3. Crear el registro de DNS apuntando al túnel
cloudflared tunnel route dns mi-app dev.mi-dominio.com

# 4. Probar a mano antes de dejarlo de servicio
cloudflared tunnel --url http://localhost:4200 run mi-app
```

Las reglas van en `%USERPROFILE%\cloudflared\config.yml`:

```
tunnel: mi-app
credentials-file: C:\Users\<tu-usuario>\cloudflared\<UUID>.json

ingress:
  - hostname: dev.mi-dominio.com
    service: http://localhost:4200
  - hostname: api.mi-dominio.com      # si quieres publicar otra pieza aparte
    service: http://localhost:8080
  - service: http_status:404          # la última regla siempre es ésta
```

Y para dejarlo permanente:

```
cloudflared service install
```

Camino 3 · El túnel como contenedor (todo en el mismo compose)

Útil si quieres que el túnel viva y muera con la aplicación:

```
tunnel:
  image: cloudflare/cloudflared:latest
  command: tunnel --no-autoupdate run --token ${CLOUDFLARE_TUNNEL_TOKEN}
  depends_on:
    portal: { condition: service_healthy }
  restart: unless-stopped
```

Aquí el servicio del panel apunta a `http://portal:8080` —el nombre del contenedor—, no a `localhost`, porque el túnel está dentro de la red de Docker.

Parte 4 · Comprobar que de verdad está publicado

```
curl https://dev.mi-dominio.com                # 200 y el HTML del portal
curl https://dev.mi-dominio.com/api/v1/health  # la API por el mismo nombre
```

Y ábrelo desde el teléfono con los datos móviles, fuera de tu red. Es la única comprobación que descarta que estés viendo tu propia máquina.

El certificado y el HTTPS los pone Cloudflare: tu aplicación sigue hablando HTTP por dentro.

Parte 5 · Publicar una versión nueva

El ciclo de siempre, cuatro pasos:

```
# 1. Guardar a dónde volver: se etiqueta la imagen que está corriendo AHORA
docker tag mi-app/portal:local mi-app/portal:rollback-20260928-1930

# 2. Reconstruir sólo lo que cambió
docker compose build portal

# 3. Recrear sólo ese contenedor. --no-deps es lo que impide que la base se reinicie de paso
docker compose up -d --force-recreate --no-deps portal

# 4. Comprobar que lo servido cambió, no sólo que compiló
curl -s https://dev.mi-dominio.com | Select-String "main-"
```

Volver atrás, si algo salió mal:

```
docker tag mi-app/portal:rollback-20260928-1930 mi-app/portal:local
docker compose up -d --force-recreate --no-deps portal
```

Dos reglas que valen oro:

- **Commit antes de publicar.** `docker compose build` toma lo que hay en disco, incluido lo que no guardaste. Si no commiteas, no hay forma de saber después qué se publicó.
- **Cambio de base de datos: respaldo verificado antes, y siempre a mano.** Nunca migraciones automáticas al arrancar el contenedor.

Parte 6 · Los cinco errores que vas a cometer

Lo que ves	Qué es	Arreglo
<code>Error 502</code> en el dominio	El túnel vive pero atrás no hay nadie: los contenedores están apagados	<code>docker compose up -d</code> , y ponles <code>restart: unless-stopped</code>
El sitio se cae al reiniciar la computadora	Sin política de reinicio	<code>restart: unless-stopped</code>
Imágenes, mapas o tipografías de otro sitio no cargan, y en desarrollo sí	La política de seguridad de nginx sólo permite tu propio origen	Agrega el origen a <code>img-src</code> / <code>font-src</code> / <code>connect-src</code>
El login se frena para todos a la vez, o no se frena nunca	La aplicación ve la dirección de nginx, no la del visitante	Confía en <code>X-Forwarded-For</code> de tu proxy, o usa <code>CF-Connecting-IP</code>
Cambiaste el código y el dominio sigue igual	Reconstruiste pero no recreaste el contenedor	El paso 3 de la Parte 5

Resumen en una pantalla

1. Levantar

```
docker compose build; docker compose up -d; docker compose ps
```

2. Dominio: comprarlo en Cloudflare, o apuntar los servidores de nombres ahí

3. Túnel

```
cloudflared.exe service install <TOKEN>
```

y en el panel: Public Hostname -> dev + mi-dominio.com -> HTTP localhost:4200

4. Comprobar desde fuera

```
curl https://dev.mi-dominio.com
```

5. Publicar: etiquetar, construir, recrear, comprobar por el dominio